Software Requirement Specifications
Wildfire Spread Prediction

Group 17
Version 1

Personnel and Roles

Team Member/Email	Role
Tom Overall - overallt@mcmaster.ca	Backend/Machine Learning Development
Selim Abdelrazek - abdels20@mcmaster.ca	Data/Machine Learning Development
Om Patel - patelo11@mcmaster.ca	Full Stack Development
Umar Rajguru - rajguruu@mcmaster.ca	Data/Machine Learning Development
Kevin Lao - laok2@mcmaster.ca	Quality Assurance and Data Development
Soutrik Guhathakurta - guhathas@mcmaster.ca	Backend/Machine Learning Development
Neel Joshi - joshin10@mcmaster.ca	Project Manager/Frontend Development

Abbreviations, Notations, Naming Conventions and Definitions

VIIRS	Visible Infrared Imaging Radiometer Suite (VIIRS)
NDVI	Normalized Difference Vegetation Index
EVI	Enhanced Vegetation Index
HDF5	Hierarchical Data Format 5

Table of Contents

Personnel and Roles 1
Abbreviations, Notations, Naming Conventions and Definitions 1
Team Member and Contributions 2

Purpose of the Project **3**
The Client and Stakeholders **3**
Project Constraints **4**
Functional Requirements **5**
Data and Metrics **7**
Non-Functional Requirements **11**
Risks and Issues Predicted **12**
Team Meeting and Communication Plan **12**
Team Member Roles **13**
Workflow Plan **13**
Requirement Set Fulfillment 14
Proof of Concept Demonstration Plan **14**
Technology **15**
Frontend (Web UI) 15
Backend (Services, Data & ML) 15
Training/Development of Model 16
Project Scheduling Gantt Chart 16

Team Member and Contributions

Team Member	Document Contribution
Tom Overall	Team meeting and Communication plan, Team Member Roles, Backend and Data storage
Selim Abdelrazek	Purpose of the Project, Client and Stakeholders
Om Patel	Frontend, Gantt Chart
Umar Rajguru	Non-functional requirements, Risk and assessments
Kevin Lao	Purpose of project, Project constraints, Proof of Concept plan
Soutrik Guhathakurta	Backend system design, Data processing
Neel Joshi	Data and Metrics, Backend Architecture/UML

Purpose of the Project

The purpose of this project, constraints barred, is to ultimately deliver a tool that predicts the spread of wildfires with sufficient accuracy. Wildfires have become an increasingly

concerning phenomenon, particularly with the growing impact of climate change on ecological stability, so devising novel approaches to their containment seems imperative.

The current paradigm for wildfire mitigation seems to revolve around deterministic/mechanistic approaches. For instance, Canada's Fire Weather Index and Fire Behavior Prediction systems do deliver reasonably effective danger assessments, but they are empirically driven and are not designed to accommodate the more peripheral factors at play such as topography, vegetation and evolving climate patterns.

As such, our goal is to leverage contemporary computational modalities to produce more sophisticated inferences for wildfire spread. We hope that by utilizing a multimodal dataset with modern machine learning approaches, we may produce a model that surpasses the predictive capacity of existing mechanistic systems such as those mentioned previously. We may also generate additional insights using statistical methodologies (see Functional Requirements) that would empower clients and/or stakeholders to more effectively obviate wildfire catastrophes.

The Client and Stakeholders

The project is not strictly intended for a specific client, nor does it fall under the direction of a specific supervisor. In the interest of moving the project toward a tangible goal and guiding its design, we may speculate as to the hypothetical ideal end-user.

In a more academic sense, we expect that the end product may be of interest to wildfire/natural disaster researchers. It may be out of the scope of our undertaking to rigorously analyze and compare the efficacy of our model against existing systems, so this task may fall to interested researchers.

Practically speaking, our ideal client would be a governing body or federal branch responsible for wildfire mitigation or general natural disaster response. In Canada this would be the Canadian Forest Service (CFS); its US counterpart is the US Forest Service (USFS).

Examples of such clients and stakeholders are:

- Federal and intergovernmental agencies (ex. US National Interagency Fire Center for coordinating response and resource allocation)
- Provincial, territorial and state agencies (ex. Ontario's Aviation, Forest Fire and Emergency Services)
- Academic and research institutions (ex. University of British Columbia, engaged in climate and forestry research)
- Insurance companies (ex. Insurance Bureau of Canada, interests in quantifying wildfire risk for liabilities)

Project Constraints

Compute Constraints

The expected deployment target of the application may not have GPU resources available for inference. CAS GPU clusters cannot provide guaranteed resources, and GPU is not available for Virtual Private Servers (VPS); if deploying on a cloud target instead, a similar constraint applies, as VPS with GPU resources are expensive, and fees would likely exceed the budget of the project.

Due to the likely constraint of having only CPU inference available on the deployment target, we expect to see some performance losses (i.e. time to infer) and to require stricter concurrent use/rate limits (i.e. to prevent overloading/blocking), when compared to a deployment target with GPU resources. Evaluation of model performance may be completed on a host with GPU resources, in order to produce metrics that are more easily comparable to those of existing work.

Data and Geographical Constraints

Due to mixed availability of the source data in different geographical areas, there is a constraint on the geographical coverage of the Wildfire Spread Prediction model. Namely, we expect the maximum coverage to be of the entirety of the United States and Canada; in other regions, then, the application will not be usable for predicting wildfire spread and any other functions.

In the case of Canada, due to differing formats of data for weather, vegetation, etc, there is also a constraint on using this data as input for the model: it must be transformed and loaded to the expected format, which is that of U.S. data. The expansion of application coverage to Canadian wildfires is a goal that would take considerable effort to achieve, and additional feasibility analysis will be performed when the project is further along in development.

Deployment Mode

The application is intended to be a web application, where users can access the client through a web browser. This decision was made with consideration to the practicality of alternatives, given the expected product; the two main factors contributing to this decision were:

- **Resource Requirements**: Due to the compute and storage requirements associated with the ML model and relevant data, a standalone desktop application would likely not be a good solution: with the model running and data being stored locally, the resource requirements would be substantial. A client-server architecture would alleviate most of this local resource demand.

- **Ease of Use:** Although a Progressive Web Application (PWA) or similar form could be used for the client, there is little expected benefit to packaging it as a PWA rather than an application that is accessed through a browser. The expected complexity of the UI is low, there is minimal information that would need to be cached locally, nor much functionality

for offline use. By packaging the application as a “traditional” web app, much complexity can be avoided.

Functional Requirements

Prediction Model Functional Requirements

- [P0] Spread area mask, U.S, historic: The model will predict a wildfire spread area mask, given the input feature matrix for some active wildfire at some point in time, where the active wildfire is occurring in the contiguous United States at a time for which there is existing, pre-processed data in the dataset. The preprocessed data for this will come from the WildfireSpreadTS dataset, which covers selected U.S. wildfires from 2018 to 2021.
- [P3] Spread area mask, Canadian, historic: The model will predict a wildfire spread area mask for historic Canadian wildfires at some point in time for which there is existing, pre-processed data in the dataset. This will require recreating the WildfireSpreadTS dataset while adding Canadian data equivalent to the U.S.-specific GRIDMET source dataset. As there is not a complete Canadian dataset which has all of the required GRIDMET features in the required time range, this data would have to be synthesized from several sources, likely including the Adjusted and homogenized Canadian climate data (AHCCD) dataset. In case of missing features, these may have to be dropped from the model, or a separate model may have to be trained on the available features for Canadian coverage.
- [P3] Increased temporal granularity: The model will predict the wildfire spread area for a time step $t + 6n$, where n is some integer, given the wildfire bounding area for a time step t and associated features (weather, vegetation, topology, etc). This would modify/extend the existing baseline models mentioned in the WildfireSpreadTS paper, taking advantage of the temporal resolution (of about 6 hours) available in the dataset; no major modifications to the dataset are expected to be required.
- [P0] Statistical inference: The prediction will come with statistical conclusions, such as confidence of prediction, as well as evaluation metrics of previous predictions
- [P1] Separating static and dynamic features: The dataset contains several features that do not see much change on a frequent basis. These features can be treated as static to allow for further optimization (see Eddin et al.).
- [P3] Optimization: As a second order priority, we may aim to implement more sophisticated optimization strategies for high-dimensional data to improve upon the issues encountered in the original WildfireSpreadTS paper.
- [P2] Next n-day Prediction: If for any day the model outputs a probability over a certain threshold t deemed “very likely”, the prediction itself can be used to model another day after providing an interval of confidence.

- [P2] Noisy labels: The acquisition process for the data was impacted by various noise sources. There is therefore room for improvement via methods that are tolerant toward label noise such as Song et al., 2022; Englesson et al., 2021.
- [P2] Uncertainty estimation: Approaches such as Gawlikowski et al., 2022 or generative modeling may be utilized to circumvent the stochasticity inherent to the process of fire spreading – particularly at the level of granularity we are working with.
- [P3] Self-supervised pretraining: As mentioned in the WildfireSpreadTS paper, SSL may prove useful for rectifying the optimization difficulties encountered.

Backend Specific Functional Requirements

- [P0] Prediction request handling: FastAPI must expose an endpoint for the backend to accept prediction requests from the frontend. Endpoint must provide all feature inputs needed for the model.
- [P0] Data preprocessing: Apache Spark pipeline preprocesses data. For example, time series data collected is resampled to consistent time intervals, missing values and outliers are handled, and features are embedded into tensor for ML input.
- [P0] Model inference: Run pre-trained model to compute predictions in parallel across partitions. Generates fire spread masks and probability tables.
- [P1] Caching of Wildfire features: For any specific wildfire certain data such as terrain and aspect/slope data does not need to be re-queried every time a prediction needs to be made. By caching these in Postgres, redundant API/data retrieval calls are avoided.
- [P2] Error persistence/recovery: Set up background scripts that can recover a model's training state and data preprocessing in case of crashes/issues. For example, during data preprocessing, there should be a temporary cache in case one step fails.
- [P2] Authentication/Access levels: Instead of exposing all endpoints publicly on a server, restrict access to only services that need them.
- [P3] Extraction and availability of new data: The application server must extract, process, and save data from external sources (e.g. NASA, NOAA) at regular intervals (or on user request), to make data from new time steps (roughly present time) available as input for predictions. This will require scheduled calls to several APIs to request the new, raw data, with sources including NASA, GRIDMET, and others. The raw data must then be processed by the server's data pipeline to be made available for new predictions.

Frontend Functional Requirements

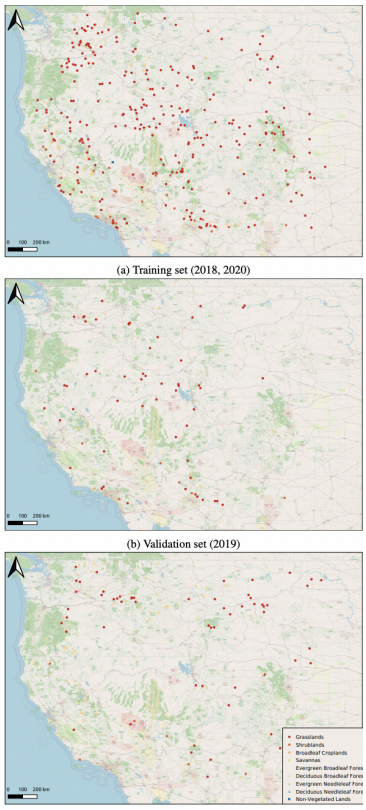
- [P0] Map view: The user will select a wildfire event from a dropdown menu where they can then visualize a wildfire over time. The wildfire will be displayed with a map overlay and the fire will be shown on a heatmap relative to time elapsed.
- [P2] User-input wildfires: The application will allow users to input a new wildfire, minimally by providing a bounding polygon mask of the active fire, from which the next day's (24 hours elapsed) spread will be predicted and displayed. At the given time step,

the other input features will be supplied either by the existing data from the WildfireSpreadTS dataset (if available at the time step), or by cached, pre-processed "new" data.

- [P3] Time scale selection: The user will have the ability to change the time frame of the wildfire being displayed. For example, they can tweak the time scale to see the predicted wildfire spread every 6, 12, or 24 hours. This feature requires the implementation of increased temporal granularity in the model, and support for predictions of different time scales on the backend.
- [P3] Event management: The application will display concurrent fire events on a map for a given time step and scale (e.g. 2020-02-01, 1-week) with high-predicted-spread fires highlighted. This would theoretically serve to support prioritization of investigating and mitigating the fire events, especially with roughly real-time "new" data. From a map view of a country or continent, then, the fires that may present the most danger at a given time are alerted to the user through indicators overlaid on the map.

Data and Metrics

This project will leverage the multimodal time series wildfire data set, WildfireSpreadTS to train and develop the wildfire prediction model https://doi.org/10.5281/zenodo.8006177.



(a) Training set (2018, 2020)

(b) Validation set (2019)

The WildfireSpreadTS dataset synthesizes several source datasets, which are comprised primarily of satellite data products; these are VNP14IMG, VNP09GA, VNP13A1, SRTMGL1, MCD12Q1.061, GRIDMET, and NOAA_GFS0P25. Each source dataset contributes one or more features, which are compiled together by the geographical location and time of recording. This data is then compared to records in the GlobFire dataset, which tracks global fires based on the MCD64A1 satellite data product. The resulting WildfireSpreadTS dataset contains samples of wildfire and non-wildfire events in the United States of America, from 2018 to 2021, where each sample contains 21 features pertaining to weather, topology, infrared, and other metrics; these data are used for training the model and to perform evaluation and inference.

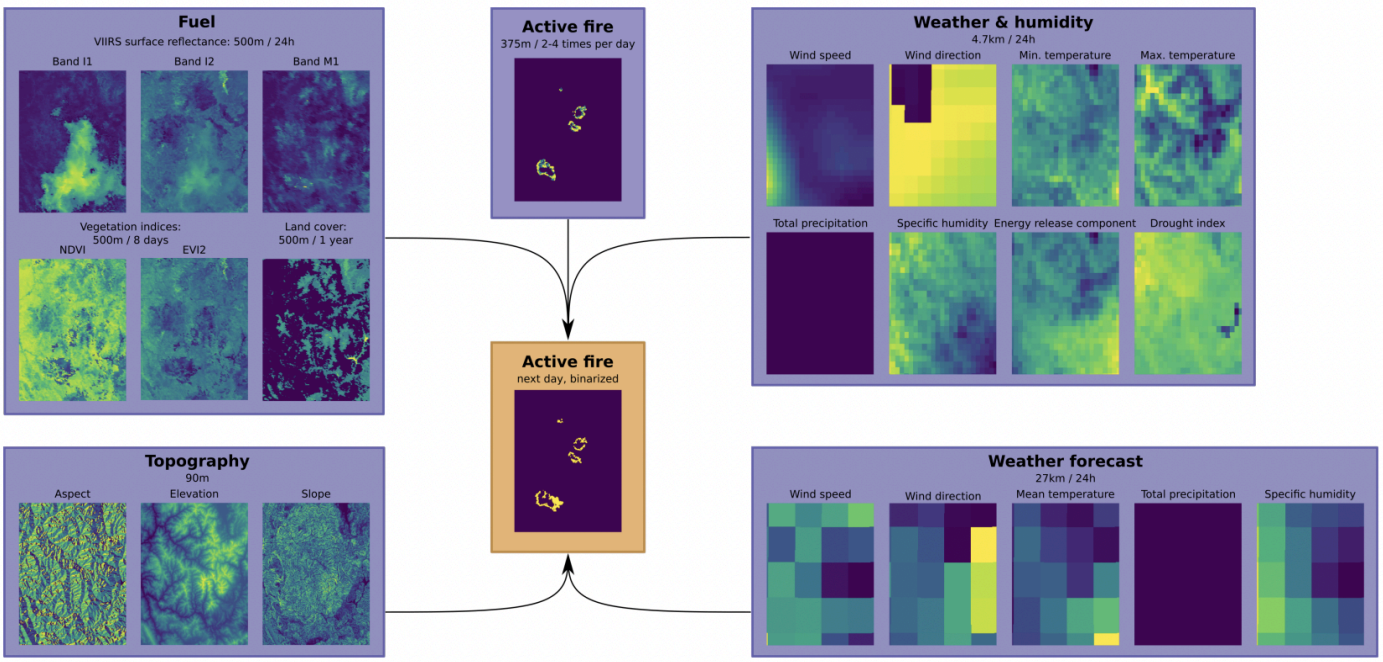
Source	Features
VNP14IMG Schroeder et al. [2014]	VIIRS active fire
VNP09GA Vermote et al. [2016]	VIIRS surface reflectance
VNP13A1 Didan and Barreto [2018]	VIIRS vegetation indices
SRTMGL1 v003 JPL [2013]	Elevation
MCD12Q1.061 Friedl and Sulla-Menashe [2022]	Land cover classification
GRIDMET Abatzoglou [2013]	Wind, rain, temperature, humidity, drought index, energy release component
NOAA_GFS0P25 Clough et al. [2005]	Forecast for wind, rain, temperature, humidity

Features

1. VIIRS (Visual, Infrared Imagery of Area, measured in Radiance or Reflectance)
2. NDVI (Vegetation and biomass Imagery of Area, index ranges from -1 to 1)
3. EVI2 (Vegetation quantity and activity of Area, index ranges -1 to 1)
4. Total Precipitation
5. Wind Speed
6. Wind direction
7. Minimum temperature
8. Maximum temperature
9. Energy release component (Numerical, related to BTU of fuel in unit area)
10. Specific Humidity (Ratio of water vapor mass in air parcel to total mass of moist air)
11. Slope
12. Aspect (Direction terrain faces)
13. Elevation
14. PDSI (Monitors drought severity)
15. Landcover type
16. Total precipitation
17. Forecast: Wind speed

18. Forecast: Wind direction
19. Forecast: Temperature
20. Forecast: Specific humidity
21. Active fire (Area mask)

Note that all relevant geospatial data is scaled to 375m to pixel. All standard non-indexed features use their standard metric units scaled to a 24h average unless otherwise specified (i.e Max temperature involves taking max instead of average).



The outer boxes show the input observations and features and the upper middle box depicts the active fire map which is updated 4 times a day via satellite. The lower middle box shows the next day's predicted wildfire map given by our model.

An important thing to note is that there exists some missing features for certain wildfires in the dataset provided. This is due to downtime of satellite imagery in that time interval or weather conditions that made relevant sensors take unreliable measurements. The table below outlines these discrepancies. To address this, these data points can be removed and only those without missing features can be kept. Another possibility is in the case of missing static features, to use from other datapoints. For example, if the slope measurements of terrain containing a wildfire is missing, it can be replaced by measurements from another wildfire that has occurred in the same area.

Feature	Missing values (%)
VIIRS band M11	4.95
VIIRS band I2	4.95
VIIRS band I1	4.95
NDVI	8.35
EVI2	8.35
Total precipitation	1.08
Wind speed	1.08
Wind direction	1.08
Minimum temperature	1.08
Maximum temperature	1.08
Energy release component	1.08
Specific humidity	1.08
Slope	0.99
Aspect	0.97
Elevation	0.97
PDSI	1.50
Landcover type	0.0
Forecast: Total precipitation	0.0
Forecast: Wind speed	0.0
Forecast: Wind direction	0.0
Forecast: Temperature	0.0
Forecast: Specific humidity	0.0
Active fire	99.83

For evaluation of the model, the test set average precision (AP) metric will be used. This single metric is calculated using the Confusion Matrix, Intersection over Union, Recall, and Precision metrics. Given these, a Precision-Recall curve is generated from which the area under the curve gives the AP value. The metric is relevant here because the dataset is very imbalanced; for a given region and satellite imagery, the pixels that represent active fires are very low. Also, metrics such as F-1 rely on a single threshold t. AP metric considers how the model performs across all thresholds and doesn't commit to a certain t value. Baseline models such as logistic regression and persistive baselines yield an AP between 0.28-0.32 for similar tasks. The expectation is to have an improvement on this range.

Model	Input days	Features			Parameters (All)
		Vegetation	Multi	All	
Persistence	1	0.193 ± 0.065	0.193 ± 0.065	0.193 ± 0.065	0
Log. Regression	1	0.279 ± 0.092	0.288 ± 0.091	0.286 ± 0.092	361
Res18 U-Net	1	0.328 ± 0.090	0.341 ± 0.085	**0.341 ± 0.086**	14.4M
Res18 U-Net	5	0.333 ± 0.079	0.344 ± 0.076	0.325 ± 0.108	14.7M
ConvLSTM	5	0.306 ± 0.082	0.310 ± 0.085	0.292 ± 0.094	240K
UTAE	5	**0.372 ± 0.088**	**0.350 ± 0.113**	0.321 ± 0.135	1.1M

For detailed baseline results, the above table outlines AP score +/- Standard deviation with 12-fold cross validation over years. Multi-feature includes vegetation, land cover, topography and weather features.

Non-Functional Requirements

Look and Feel Requirements

The webpage must present a clean, modern, and intuitive interface that employs a consistent and seamless user experience throughout its use and navigation. There will be a unified and agreed upon colour palette, typography, and iconography creating a coherent visual identity. Consistent page component styling will be enforced, ensuring uniform appearance and behaviour of buttons, sliders, drop-down menus, and all other page elements.

Usability and Humanity Requirements

In order to ensure clarity, accessibility and ease of use for all users with disabilities, the page will be friendly to assistive technologies such as screen readers and magnifying tools. All buttons and interactive tools will have clear feedback mechanisms with error and confirmation messages. Tooltips will be added to all areas where clarification would be helpful.

Performance and Speed Requirements

In order to ensure a fast and responsive UI, quick page loads, smooth map visualizations, most computations should be done server side and responses should be packaged concisely. This way, under varying network conditions and client CPU speeds, the frontend experience is consistent. Servers will also have disaster recovery plans to allow uptime even in the event a cluster or one of its services encounters an issue or error. All services will be continuously monitored to ensure all operations perform as intended.

Security and Privacy

All sensitive data must be encrypted including API keys used to obtain data from external resources as well as user authentication. All API endpoints will be exposed to only those services that need them and data retrieval will only be permissible by approved services. User inputs will be validated and sanitized to prevent arbitrary code execution and injection. Packages, modules and third party dependencies will be monitored and updated to address any security risks or vulnerabilities. In order to protect servers against DDoS and malicious traffic, rate limit and continuous monitoring via CloudFlare will be used.
All traffic will be logged and audited to aid in incident response and used to detect any suspicious activity. Finally, end-user license agreements will be available to allow users to understand how their data is stored and used and limit liability related to prediction accuracy.

Legal

The training/test dataset, use of new data, and application itself must respect the associated licenses and legal obligations from data sources and software regulations.

Additionally, it must be made clear that the application and all source code are created for an academic project, where no guarantees are made regarding accuracy and liability. A disclaimer and end user license agreement must be displayed on the client webpage before the application can be used, so that this information can be clearly presented to prospective users and agreed to.

Risks and Issues Predicted

Risks in the Wildfire Spread Prediction project primarily involve performance, data, and compliance challenges. Latency and inference time present technical risks, as slow model responses could delay the delivery of wildfire spread forecasts during active events. Real-time performance is critical; even minor prediction lags may hinder emergency decision-making or reduce the model's operational value. Ensuring low-latency data processing and leveraging GPU acceleration will be essential to maintain timely, actionable predictions.

A second major concern is data coverage and reliability. Limited availability of additional wildfire data could reduce model accuracy across diverse terrain and vegetation types. Many global datasets focus on other regions, potentially creating bias in predictions. Continuous access to high-quality, up-to-date data—such as MODIS, VIIRS, and Sentinel imagery—is also vital. Interruptions or inconsistencies in these feeds could degrade the model's accuracy over time, so redundant data pipelines and automated monitoring will be necessary to maintain reliability.

Finally, legal and regulatory compliance must be managed carefully. The project relies on multiple third-party and open-source datasets, each with its own licensing conditions and data-use restrictions. Compliance with data privacy and institutional guidelines is required to avoid potential violations or liabilities. Proactive mitigation through optimized infrastructure, diverse datasets, and strict adherence to licensing will ensure that the system remains reliable, ethical, and trustworthy for real-world wildfire forecasting.

Project Development Plan

Version 0

Team Meeting and Communication Plan

Team meetings are held bi-weekly to assign new tasks, and discuss upcoming milestones. Additional sessions are scheduled as needed. A rotating member maintains the meeting agenda, outlining discussion topics and their priority. Linear serves as the central hub for workflow and task tracking, while GitHub manages version control, project files, and meeting notes. Google Docs is used for brainstorming and shared documentation, and Discord functions as the primary communication channel for discussions, updates, and coordination before tasks are formalized in Linear.

Team Member Roles

Team Member	Functional Requirements	Non-Functional Requirements
Tom Overall	Prediction handling, preprocessing, inference, authentication, feature caching	Security and privacy
Selim Abdelrazek	Statistical inference, uncertainty estimation, spread mask, error recovery	Performance and speed, Security and privacy
Om Patel	Map view, spread mask, new wildfire events, time-scale control, frontend integration	Usability and humanity requirements, Cultural requirements, Maintainability and support
Umar Rajguru	Spread mask (U.S., historic), static/dynamic feature separation, noisy labels, feature caching	Performance and speed optimization
Kevin Lao	Preprocessing, inference, error recovery, noisy labels, authentication, data extraction	Security and privacy, Legal, Performance and speed
Soutrik Guhathakurta	Preprocessing, feature caching, authentication, error recovery, spread mask (Canada), optimization	Performance and speed, Legal
Neel Joshi (Project Manager)	Spread mask (Canada, historic), statistical inference, temporal granularity, noisy labels, self-supervised training	Compliance, Legal, Look and feel

Workflow Plan

Our team uses GitHub for version control, with each member working on feature branches and submitting pull requests for peer review before merging. GitHub Issues track development tasks, linked directly to Linear items for workflow integration.

All raw data will be stored on a shared CAS server with access to the GPU cluster allowing all members to access and preprocess data using standardized scripts. Heavy compute tasks, including model training, will run on the GPU cluster. For deployment, the model and application components will be hosted preferably on CAS VPS.

Requirement Set Fulfillment

1. **Prediction Model:** Apache Spark will facilitate the environment for batch training and predicting. WandB will be used for fine tuning and evaluating on test data. All scripts will be written in Python
2. **Backend:** PostgreSQL ("Postgres") will be the main DB used for data storage. FastAPI with StrawberryGraphQL will be used to create API routes for the frontend and to interact with Apache Spark and the database/cache.
3. **Frontend:** All components will be built using Typescript with React and served with Vite/Nginx. Leaflet will be used to provide a geographic map and for visualization of fire spread.
4. **Infrastructure:** Single-host deployment on CAS server using Docker + Compose for service orchestration (Postgres, API/BFF, Nginx). CI/CD via GitHub Actions builds and pushes images.

Proof of Concept Demonstration Plan

The Proof of Concept (PoC) will demonstrate a minimal web app that accepts a wildfire input, performs model inference on the server, and returns the predicted next-day spread mask for display on the UI. This requires developing three core components.

- **Prediction Model**: A machine learning model trained on the WildfireSpreadTS dataset, which, given the input of a wildfire event (fire area mask at time step t), and its corresponding weather/vegetation/topology data etc., will predict the fire area mask at time step t + 24h.
- **Backend**: A server process that receives requests from the client regarding a wildfire event, query the model to perform prediction, then respond to the client with the predicted mask to be rendered.
- **Frontend**: An interactive web interface that displays a geographic map with options to select wildfire events and to request predictions, then receive predictions from the server that are rendered on top of the geographic map to show the predicted fire spread.

Technology

Frontend (Web UI)

The frontend will be built with TypeScript and React, chosen for its component-based architecture and strong state management with Redux. Development will take place in Visual Studio Code, using Vite for fast builds and live reloading. The production build will be served via Nginx for SSL, load balancing, and caching. Application state and cookies will be managed through Redux for consistency, and the UI will use Material UI for a clean, accessible design system.

Testing Framework and Quality Assurance

The frontend will follow a unified testing and quality assurance strategy to ensure reliability and maintainability. Vitest (or Jest) with React Testing Library will validate UI behavior such as map layer toggles, button interactions, and state updates. Playwright will perform end-to-end testing, simulating full user flows like loading fires, adjusting time intervals, and toggling basemaps. Code quality and coverage will be enforced using ESLint and TypeScript strict mode, maintaining near-100% test coverage across core logic and UI components.

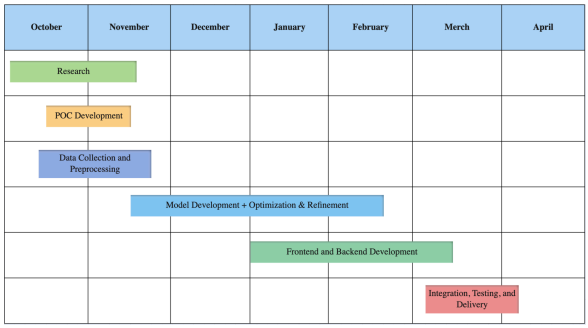
Backend (Services, Data & ML)

The backend will use FastAPI, Apache Spark, to store and fetch from the GraphQL and Postgres DBs (with extensions) and query the model for inference. The pipeline will look as follows

1. Client initiates a request to FastAPI/GraphQL for a prediction that contains all necessary inputs to model as request parameters
2. If there exists no cached or stored result for the prediction request, the request is forwarded to Apache Spark which will preprocess input data and set up a directory to save checkpoints.
3. Apache Spark will also query Postgres_DB to obtain static features such as terrain, aspect, slope, etc.
4. Postgres must return these requested features. If features do not exist, external APIs must be queried to obtain the raw data (e.g. https://api.nasa.gov/)
5. Apache Spark now starts model prediction after conforming all inputs into a valid tensor. Checkpoints are saved and reloaded in case of error or accidental termination.
6. After prediction is completed, it is emitted concurrently to the frontend (via GraphQL and Postgres for visualization and metric evaluation (i.e determining confidence bound of the prediction)

Visual Studio Code will be used at the primary coding environment for the backend.

Training/Development of Model

The dataset is built in Google Earth Engine, combining multiple source datasets to produce all required features. It is exported in HDF5 format for faster training. The Python environment is defined by a requirements.txt file, with configuration files in cfgs/. Weights & Biases (WandB) manages hyperparameter tuning via grid search and logs experiment runs. PyTorch Lightning handles the training, validation, and testing loops, integrating directly with WandB for automated logging and reproducibility. The CAS servers contain four H100 GPUs that will be used to train the model. PyTorch, Scikit learning and Numpy will be used to write training/validation scripts.

Project Scheduling Gantt Chart